Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251229**
Nama Lengkap	**Ignatius Harya Nugraha**
Minggu ke / Materi	**06 / Struktur Kontrol Perulangan**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Definisi Perulangan

Perulangan(loop) merupakan salah satu struktur kontrol utama dalam pemrograman. Dalam Python, jalannya program dapat diatur secara sekuensial, percabangan, perulangan, maupun kombinasi dari ketiganya. Secara umum, perulangan digunakan ketika program perlu:

- Melakukan suatu hal yang sama beberapa kali secara berulang-ulang.
- Melakukan suatu hal secara bertahap, di mana setiap tahap sebenarnya memiliki langkah yang sama.
- Mengakses sekumpulan data dalam suatu struktur data, misalnya List, Tuple, Queue, Stack, dan beberapa struktur data lainnya

Dalam penerapannya perulangan pada kehidupan sehari hari dapat kita lihat pada saat seorang guru yang harus mengabsen 30 siswa di kelas. Proses yang dilakukan adalah proses yang terus berulang yaitu memanggil nama dan menunggu respons untuk setiap siswa. Inilah gambaran dari perulangan dalam pemrograman.

Pada Python, perulangan dapat dilakukan dengan menggunakan for, while, maupun dengan cara rekursif. Pada laporan praktikum pertemuan ini, akan dibahas mengenai perulangan for dan while beserta kontrol perulangan menggunakan break dan continue.

Perulangan For

Perulangan for pada Python biasanya digunakan pada kondisi di mana jumlah perulangan sudah diketahui sejak awal, atau perulangan terjadi karena operasi yang sama pada suatu rentang data atau rentang nilai tertentu. Perulangan for pada rentang tertentu lebih mudah dilakukan dengan menggunakan fungsi range().

Fungsi Range

Fungsi range() pada Python memiliki beberapa bentuk penggunaan, yaitu:

- **range(stop)** Ini berarti range akan menghasilkan rentang dari 0 sampai stop-1. Contoh: range(6) menghasilkan 0, 1, 2, 3, 4, 5.

- **range(start, stop)** Dengan tambahan start maka akan menghasilkan rentang dari start sampai stop-1 dengan langkah default 1.
- **range(start, stop, step)** Tambahan step dibelakang akan menghasilkan rentang dari start sampai stop-1 dengan langkah sejumlah step (bisa positif maupun negatif).

Contoh Penggunaan

Berikut adalah contoh program untuk menampilkan bilangan dari 1 sampai 100:

```
for i in range(1, 101):
    print(i)
```

Pada program di atas, variabel i berfungsi sebagai counter yang nilainya naik secara berurutan sesuai nilai yang dihasilkan fungsi range(1, 101), yaitu mulai dari 1 sampai 100. Jika counter tidak diperlukan, dapat digunakan underscore (_) sebagai pengganti variabel counter:

```
for i in range(100):
    print('Hello World')
```

Program tersebut akan mencetak teks 'Hello World' sebanyak 100 kali tanpa membutuhkan nilai counter.

Step Negatif

Fungsi range() juga dapat menerima step negatif, yang berguna ketika diperlukan perulangan dari nilai besar ke nilai kecil. Berikut contoh menampilkan bilangan genap dari 100 sampai 2:

```
for i in range(100, 1, -2):
    print(i)
```

Program tersebut menampilkan bilangan genap mulai dari 100, 98, 96, 94, ..., sampai 2

Perulangan While

Berbeda dengan for, perulangan while umumnya digunakan pada kondisi di mana jumlah perulangan belum diketahui sebelumnya. Perulangan while akan terus berjalan selama kondisi yang ditetapkan masih bernilai True. Struktur umum perulangan while adalah sebagai berikut:

```
kondisi=True
while kondisi== True:
    print("Hello World")
    kondisi=False
```

Tiga hal penting yang harus ada dalam perulangan while adalah: nilai awal (untuk memulai perulangan), kondisi akhir (untuk menentukan kapan perulangan berhenti), dan langkah/step (agar iterasi dapat berjalan dari nilai awal hingga mencapai kondisi akhir).

Contoh Penggunaan while

Berikut adalah contoh perulangan while untuk memastikan input yang dimasukkan oleh pengguna adalah bilangan genap:

```
bilangan = 0
genap = False
while genap == False:
    bilangan = int(input('Masukkan bilangan genap: '))
    if bilangan % 2 == 0:
        genap = True
print(bilangan, 'yang anda masukkan adalah bilangan genap')
```

Program di atas terus meminta input hingga pengguna memasukkan bilangan genap. Kasus ini sangat sesuai menggunakan while karena tidak diketahui berapa kali pengguna akan memasukkan nilai yang salah sebelum akhirnya memberikan bilangan genap.

Break dan Continue

Perulangan dapat dikontrol lebih lanjut menggunakan dua kata parameter khusus yaitu break dan continue. Keduanya memiliki fungsi yang berbeda dan digunakan pada kondisi yang berbeda pula.

Break

Break digunakan untuk menghentikan perulangan secara paksa sebelum kondisi berakhir secara alami. Ketika Python menemukan instruksi break, eksekusi langsung keluar dari blok perulangan dan melanjutkan ke baris kode setelah perulangan tersebut.

Berikut contoh penggunaan break untuk menghentikan perulangan saat nilai i mencapai 5:

```
for i in range(1, 11):
    if i == 5:
        break
    else:
        print(i)
print('Selesai')
```

Saat nilai i = 5, kondisi i == 5 menghasilkan True sehingga instruksi break dijalankan. Perulangan langsung dihentikan dan program melanjutkan ke baris print('Selesai'). Dengan demikian output yang dihasilkan hanya angka 1, 2, 3, 4, kemudian 'Selesai'.

Continue

Continue digunakan untuk melewati satu iterasi dan langsung melanjutkan ke iterasi berikutnya tanpa menghentikan keseluruhan perulangan.

Berikut contoh menampilkan angka 1 sampai 10 namun melewatkan angka 6:

```
for i in range(1, 11):
    if i == 6:
        continue
    else:
        print(i)
print('Selesai')
```

Ketika i = 6, instruksi continue menyebabkan Python melewatkan sisa kode dalam iterasi tersebut (termasuk print(i)) dan langsung melanjutkan ke iterasi berikutnya (i = 7). Hasilnya, angka 6 tidak muncul dalam output.

Konversi bentuk perulangan for menjadi while

Sebagian besar perulangan for dapat dikonversi menjadi bentuk while. Untuk melakukan konversi, kita perlu mengidentifikasi tiga komponen utama dari perulangan for: nilai awal (start), kondisi akhir (stop), dan langkah (step).

Sebagai contoh, perhatikan perulangan for berikut:

```
for i in range(1, 11):
    print(i)
```

Dari perulangan tersebut dapat diidentifikasi:

- Nilai awal: i = 1 (argumen start pada range)
- Kondisi akhir: i <= 10 (argumen stop - 1 pada range)
- Langkah: i = i + 1 (argumen step default pada range)

Maka konversi ke bentuk while menghasilkan:

```
i = 1            Sebagai nilai awal
while i <= 10:   Kondisi saat perulangan berakhir
    print(i)
    i = i + 1    Langkah yang dilakukan saat perulangan
```

Kedua program tersebut menghasilkan output yang persis sama, yaitu mencetak bilangan 1 sampai 10. Perbedaan utamanya adalah pada for, manajemen counter diatur otomatis oleh range(), sedangkan pada while kita harus mengelola counter secara manual.

BAGIAN 2: LATIHAN MANDIRI (60%)

Latihan 6.1

Buatlah program yang menerapkan perhitungan perkalian dengan menggunakan penjumlahan. Buatlah fungsi perkalian() dalam program tersebut! Berikut ini adalah beberapa contoh perhitungan yang diharapkan:

- 6 x 5 = 5 + 5 + 5 + 5 + 5 + 5 = 30.
- 7 x 10 = 10 + 10 + 10 + 10 + 10 + 10 + 10 = 70.

Jawab :

```python
def perkalian(a, b):
    hasil = 0
    penjumlahan = []
    for i in range(a):
        hasil += b
        penjumlahan.append(str(b))
    proses = ' + '.join(penjumlahan)
    print(f'{a} x {b} = {proses} = {hasil}')
    return hasil

a = int(input('Masukkan bilangan pertama: '))
b = int(input('Masukkan bilangan kedua: '))
perkalian(a, b)
```

Output:

```
PS D:\Project\Kuliah\PrakAlPro-71251229> cd .\Minggu_6\
PS D:\Project\Kuliah\PrakAlPro-71251229\Minggu_6> python .\Latihan6.1.py
Masukkan bilangan pertama: 1
Masukkan bilangan kedua: 2
1 x 2 = 2 = 2
PS D:\Project\Kuliah\PrakAlPro-71251229\Minggu_6> python .\Latihan6.1.py
Masukkan bilangan pertama: 3
Masukkan bilangan kedua: 7
3 x 7 = 7 + 7 + 7 = 21
```

Penjelasan:

Karena perkalian adalah penjumlahan berulang. artinya menjumlahkan angka a sebanyak b kali. Fungsi ini melakukan persis hal itu dengan loop for, dengan menyimpan prosesnya untuk ditampilkan.

Latihan 6.2

Buatlah program yang dapat menampilkan deret bilangan ganjil dari batas bawah dan batas atas yang diberikan oleh pengguna. Jika ternyata batas atas < batas bawah, berarti deret tersebut dimulai dari batas atas, sampai batas bawah (negatif range). Buatlah fungsi ganjil() dalam program tersebut! Berikut ini adalah contoh hasil yang diharapkan:

- bawah = 10, atas = 30. Karena bawah < atas, berarti dari kecil ke besar, maka hasilnya adalah: 11, 13, 15, 17, 19, 21, 23, 25, 27, 29.
- bawah = 97, atas = 82. Karena bawah > atas, berarti dari besar ke kecil, maka hasilnya adalah: 97, 95, 93, 91, 89, 87, 85, 83.

Jawab:

```
def ganjil(bawah, atas):
    if bawah < atas:
        n = bawah + 1
        if n % 2 == 0:
            n += 1
        while n <= atas:
            print(n, end=', ')
            n += 2
    else:
        n = bawah if bawah % 2 != 0 else bawah - 1
        while n >= atas:
            print(n, end=', ')
            n -= 2

bawah = int(input('Masukkan batas bawah: '))
atas  = int(input('Masukkan batas atas: '))
ganjil(bawah, atas)
```

Output:

```
PS D:\Project\Kuliah\PrakAlPro-71251229\Minggu_6> python .\Latihan6.2.py
Masukkan batas bawah: 20
Masukkan batas atas: 30
21, 23, 25, 27, 29,
PS D:\Project\Kuliah\PrakAlPro-71251229\Minggu_6> python .\Latihan6.2.py
Masukkan batas bawah: 33
Masukkan batas atas: 42
35, 37, 39, 41,
```

Penjelasan:

Sesuai dengan soalnya karena udah jelas di soalnya.

Latihan 6.3

Buatlah sebuah program penghitung nilai Indeks Prestasi Semester (IPS). Input: jumlah mata kuliah dan nilai A/B/C/D untuk setiap mata kuliah. SKS setiap mata kuliah diasumsikan selalu 3. Bobot nilai: A=4, B=3, C=2, D=1. Output: nilai IPS yang didapatkan.

Jawab:

```python
def hitung_ips(jumlah_mk):
    bobot = {'A': 4, 'B': 3, 'C': 2, 'D': 1}
    sks_per_mk = 3
    total_bobot = 0
    total_sks = 0

    for i in range(1, jumlah_mk + 1):
        nilai = input(f'Nilai MK {i}: ').upper()
        if nilai in bobot:
            total_bobot += bobot[nilai] * sks_per_mk
            total_sks   += sks_per_mk
        else:
            print('Nilai tidak valid, dilewati.')

    if total_sks > 0:
        return round(total_bobot / total_sks, 2)
    else:
        return 0

print('Program penghitung IPS Mahasiswa')
jumlah = int(input('Berapa jumlah mata kuliah? '))
ips = hitung_ips(jumlah)
print(f'Nilai IPS anda semester ini {ips}')
```

Output:

```
PS D:\Project\Kuliah\PrakAlPro-71251229\Minggu_6> python .\Latihan6.3.py
Program penghitung IPS Mahasiswa
Berapa jumlah mata kuliah? 6
Nilai MK 1: A
Nilai MK 2: B
Nilai MK 3: B
Nilai MK 4: A
Nilai MK 5: A
Nilai MK 6: C
Nilai IPS anda semester ini 3.33
PS D:\Project\Kuliah\PrakAlPro-71251229\Minggu_6>
```

Penjelasan:

Program IPS ini menggabungkan dua konsep sekaligus: perulangan for untuk mengiterasi setiap mata kuliah, dan percabangan if-else di dalam perulangan tersebut untuk memetakan nilai huruf ke bobot angka.

Link Repository Github : https://github.com/Haigry/PrakAlPro-71251229